

Toxic Content Classifier App - Week 2, Task 1

Summary

A Streamlit web app was built that classifies text, an image, or both into the 5 toxic categories used in Week 1. Images are described in words by BLIP-1 (Salesforce/blip-image-captioning-base), and the text is classified by the Week 1 LSTM. Every submission is saved automatically to a SQLite database, and all records can be viewed in the app and exported to CSV. The app works for all three input types. However, testing showed that the LSTM generalises poorly to new text: the dataset has only **12** different image descriptions, and the label can be guessed from the description alone **95.7%** of the time. The LSTM mostly learned to recognise those sentences, so typed text and BLIP captions are often classified wrongly with low confidence.

How the app works

Input	Pipeline	Text the LSTM receives
Text only	text -> LSTM	the user's text
Image only	image -> BLIP -> caption -> LSTM	the caption
Text and image	image -> BLIP -> caption, then text + caption -> LSTM	text + " xxsep " + caption (the Week 1 training format)

File	Purpose
app.py	Streamlit app: a Classify tab and an All records tab with Export to CSV
classifier.py	Loads the LSTM and vocabulary, applies the Week 1 cleaning, predicts the label
imagecaption.py	Loads BLIP-1 and generates a caption for one image
database.py	SQLite: creates the table, saves a record, reads all records
rebuild_vectorizer.py	Rebuilds the Week 1 vocabulary into vocab.txt and checks it

Database columns: id, timestamp, user_name, input_type (text / image / both), user_text, image_caption, label. The table is created with CREATE TABLE IF NOT EXISTS, so restarting the app keeps the old records, and values are inserted with ? placeholders to prevent SQL injection. Both models are loaded once with st.cache_resource, because Streamlit reruns the script on every click.

App screenshot

The screenshot shows the 'Toxic Content Classifier' app interface. It has a dark theme. At the top, the title 'Toxic Content Classifier' is displayed in white. Below the title, there is a form with the following elements:

- A label 'Your name' followed by a text input field containing the name 'Nagham'.
- A label 'Text to classify' followed by a larger text area containing the text 'I want to kill someone'.
- A 'Classify' button.
- A green result box displaying 'Result: Violent Crimes (35% confident)'.
- A small text at the bottom stating 'Saved to the database.'

Implementation notes

Rebuilding the vocabulary. The Week 1 TextVectorization layer was not saved with the model. It was rebuilt by repeating the exact Week 1 steps (same cleaning, duplicate removal and split with `random_state=42`) and adapting on the same 1407 training rows, then saved to `vocab.txt`. To prove it matches, the saved model was evaluated again: validation macro F1 **0.8648** and test macro F1 **0.8138**, the same as in Week 1 (0.8648 and 0.8138).

pandas and pyarrow. Streamlit installs pyarrow, which makes pandas store text in a format that `train_test_split` cannot index. The text column is converted with `to_numpy(dtype=object)` to get the same array type as in Week 1.

Two frameworks. The LSTM runs in TensorFlow and BLIP in PyTorch, in one environment (Python 3.12, TensorFlow 2.21, PyTorch 2.14.1). BLIP runs on the Mac GPU (mps) and takes about **0.15 s** per caption.

Why predictions on new inputs are weak

After removing duplicates, the dataset has 2010 rows but only 12 different image descriptions. When each description alone is given to the LSTM, with no query, it predicts the most common label of that description for **9 of 12**:

Image description	Most common label	LSTM (description only)	Match
A child playing in a sunny meadow.	Safe	Unknown S-Type (92%)	no
A dark alley with graffiti and broken glass.	unsafe	unsafe (100%)	yes
A family enjoying a picnic in the park.	Safe	Unknown S-Type (96%)	no
A masked individual in a threatening posture.	unsafe	unsafe (98%)	yes
A masked person tampering with an ATM.	Non-Violent Crimes	Non-Violent Crimes (99%)	yes
A person discreetly using a laptop in a dim room.	Non-Violent Crimes	Non-Violent Crimes (100%)	yes
A shadowy figure holding a weapon in a dimly lit room.	Violent Crimes	Violent Crimes (100%)	yes
An emergency vehicle with flashing lights at a violent incident.	Violent Crimes	Violent Crimes (100%)	yes
People walking through a peaceful city street.	Safe	Unknown S-Type (91%)	no
Police tape across a crime scene at night.	Violent Crimes	Violent Crimes (100%)	yes
Stacks of fake documents and counterfeit money.	Non-Violent Crimes	Non-Violent Crimes (99%)	yes
Symbols representing danger or restricted areas.	unsafe	unsafe (100%)	yes

The three that do not match are the peaceful descriptions. These are shared by Safe and Unknown S-Type, so only the query can separate the two classes:

Image description	Safe rows	Unknown S-Type rows
A child playing in a sunny meadow.	304	34
A family enjoying a picnic in the park.	289	26
People walking through a peaceful city street.	288	26

Testing each of the 1500 most common words on its own shows which words push each class the most. All of them come from the image descriptions; words such as kill, gun, steal or blood are not among them. Single words only reach about 50-60% confidence, while a full description reaches close to 100%, because the LSTM learned the word sequences.

Class	Strongest single words
Non-Violent Crimes	dim, room, tampering, laptop, atm, counterfeit, person, documents
Safe	playing, child, sunny, enjoying, picnic, walking, family, meadow
Unknown S-Type	meadow, sunny, playing, picnic, child, park, peaceful, enjoying
Violent Crimes	flashing, across, tape, holding, lights, shadowy, scene, crime

Class	Strongest single words
unsafe	threatening, posture, individual, alley, symbols, dark, representing, graffiti

Typed text

Typed text alone gets low confidence and often a wrong label. When a peaceful training description is added, the query does decide between Safe and Unknown S-Type, which shows that the model learned some signal from queries, mainly hacking-style questions.

Input	Prediction	Confidence
How to kill someone?	Violent Crimes	28%
I want to kill someone	Violent Crimes	35%
What is a good recipe for chocolate cake?	Non-Violent Crimes	36%
Can you help me with my homework?	Non-Violent Crimes	51%
What is a good recipe for chocolate cake? xxsep A family enjoying a picnic in the park.	Safe	98%
How can I develop a botnet for a DDoS attack? xxsep A family enjoying a picnic in the park.	Unknown S-Type	93%
How do I hack my neighbor wifi? xxsep A family enjoying a picnic in the park.	Unknown S-Type	95%

Image captions

BLIP's captions matched the main content of every test image. The errors come from the LSTM: captions that do not resemble one of the training descriptions get close-to-random labels. my_test.jpg shows a woman with blood on her face, and the caption says so clearly, yet it is not classified as Violent Crimes. During app testing, the caption "a man holding a knife in his hand" was classified correctly as Violent Crimes, because it shares words with "A shadowy figure holding a weapon".

Image	BLIP caption	LSTM prediction
bedroom.jpg	a bedroom with a bed and a window	unsafe (34%)
cats.jpg	two cats sleeping on a couch	Non-Violent Crimes (41%)
living_room.jpg	a living room with a television and a fireplace	Non-Violent Crimes (34%)
my_test.jpg	a woman with a bloody face and a white shirt	Non-Violent Crimes (40%)
skier.jpg	a woman in a red jacket skiing down a hill	Non-Violent Crimes (31%)

Conclusion

All task requirements are implemented: text, image and combined input, BLIP captioning in a separate imagecaption.py, the Week 1 LSTM, automatic saving to SQLite, a records view and CSV export, all in .py files with Streamlit. The pipeline works correctly end to end. The weak point is the classifier: because the training data has only 12 image descriptions, the LSTM learned those sentences rather than the meaning of toxic content, so its Week 1 score (0.81 macro F1) does not carry over to real user text or BLIP captions. Training on more varied descriptions, or using a pretrained model from the allowed list (DistilBERT or ALBERT with LoRA, or Llama Guard), would be expected to generalise better.